

Name-surname: Cheick Moctar Traore

ID: 2592106

Course: MSCS 711 Computer Security

Machine learning-base phishing detection

Introduction

Phishing is a cyberattack that combines social engineering and technical subterfuge to deceive victims into revealing sensitive information such as passwords and financial credentials by impersonating trusted entities through fake websites and deceptive emails.

According to the Anti-Phishing Working Group (APWG) Phishing Activity Trends Report for the fourth quarter of 2024, a total of 989,123 phishing attacks were recorded, with monthly attacks ranging between 290,000 and 358,000, representing a significant increase compared to previous quarters.

The most targeted sectors in Q4 2024 were SaaS and Webmail services at 23.3%, followed by Social Media platforms at 22.5% and Financial Institutions at 11.9%, demonstrating that phishing attacks broadly target industries where users store sensitive personal and financial information (APWG, 2024).

This project aims to develop and compare machine learning-based models for automatic phishing website detection, evaluating traditional machine learning algorithms including Decision Tree, Random Forest against a deep learning approach using a Feedforward Neural Network, with the goal of identifying the most effective model for real-world phishing detection

Background & Related Work

Phishing attacks were first recorded in 1996 when attackers used randomized algorithms to steal user credentials from America Online (AOL), and have since evolved into sophisticated attacks combining social engineering and technical subterfuge that continuously adapt to bypass traditional rule-based countermeasures (Aleroud & Zhou, 2017).

Over time, phishing attacks evolved beyond simple email scams to include more sophisticated techniques such as targeted attacks against specific individuals, phone-based phishing, and the

injection of malicious programs into victims' computers, making them increasingly difficult to detect and prevent (Aleroud & Zhou, 2017).

Traditional rule-based and blacklist approaches to phishing detection proved insufficient because attackers continuously adapted their techniques in response to new countermeasures, creating new phishing URLs and domains faster than static rules could be updated to block them (Aleroud & Zhou, 2017).

Machine Learning as a Solution for Phishing Detection

Machine learning was proposed as a more effective solution for phishing detection by automatically extracting and analyzing website features without human intervention, overcoming the limitations of manual rule-based approaches that were too slow to keep up with evolving attacks (Mohammad, Thabtah & McCluskey, 2014).

Unlike static rule-based systems, machine learning models can analyze large datasets of website features such as URL characteristics, domain age, and SSL certificates to automatically discover hidden patterns that distinguish phishing websites from legitimate ones, making them more adaptable and scalable than manual approaches (Mohammad, Thabtah & McCluskey, 2014).

Previous research on ML-based phishing detection

A comparative study by Alhogail (2018) evaluated six machine learning classifiers including Logistic Regression, Decision Tree, SVM, Random Forest and Neural Networks for phishing detection, finding that Random Forest achieved the lowest error rate of 7.72% while Neural Networks performed worst, consistent with our findings on website-based phishing detection.

However, this study was limited to a small dataset of 2,889 emails and 43 features, and the authors acknowledged the need for larger datasets and automated feature extraction mechanisms to keep up with evolving phishing techniques, gaps that our project addresses by using a dataset of 88,647 websites with 111 features (Alhogail, 2018)

Dataset Description

Dataset Source and Description

This project utilizes a publicly available phishing detection dataset obtained from GregaVrbancic's GitHub repository, containing 88,647 website samples with 111 numerical features extracted from URL characteristics, domain properties, and webpage content, along with a binary target label indicating whether each website is phishing or legitimate.

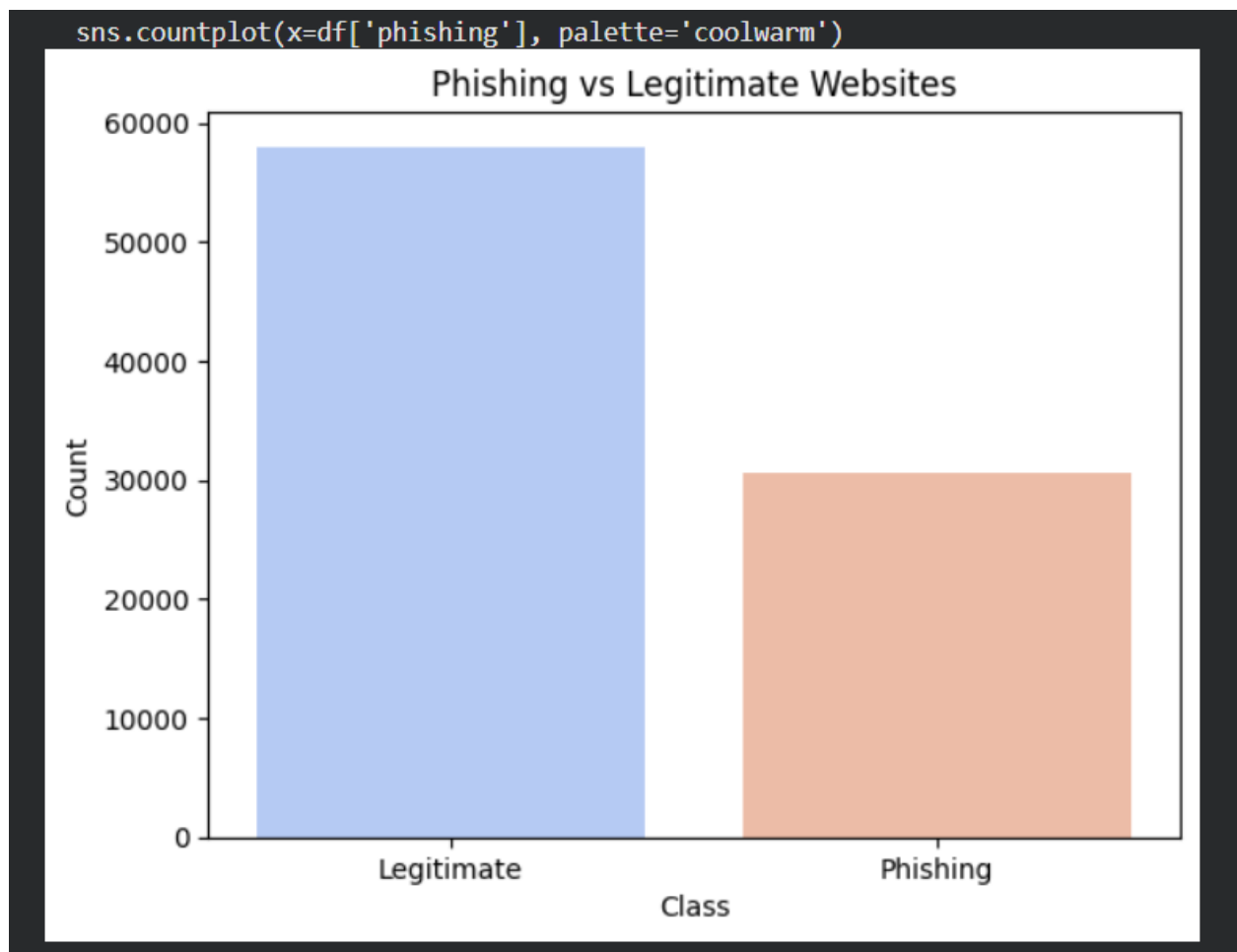
Features Explanation

The 111 features are organized into three main categories: URL-based features such as URL length, number of dots, hyphens, and special characters; domain-based features such as domain

activation time, number of nameservers, and MX servers; and page-based features such as SSL certificate presence, number of redirects, and Google index status, all of which are represented as numerical values requiring no text conversion.

Class Distribution

The dataset contains 58,000 legitimate websites (65.4%) and 30,647 phishing websites (34.6%), representing a moderate class imbalance where legitimate websites outnumber phishing websites by approximately 2:1, which was addressed during the train/test split by using stratified sampling to ensure both sets maintained the same class ratio.



Preprocessing Steps

The dataset required minimal preprocessing as all features were already in numerical format with no missing values. The data was split into 80% training (70,917 samples) and 20% testing (17,730 samples) using stratified sampling to preserve class distribution, and feature scaling was applied using StandardScaler to normalize all features to the same scale, which was essential for Neural Network models that are sensitive to feature magnitude.

Methodology

This project evaluates four machine learning models for phishing website detection, comparing two traditional machine learning algorithms against a deep learning approach, with all models trained and tested on the same dataset and evaluation conditions to ensure a fair comparison.

Model 1 : Decision Tree

A Decision Tree is a supervised machine learning algorithm that recursively partitions the feature space into subsets by selecting the feature and threshold that best separates the classes at each node, following a tree-like structure of yes/no decisions that leads to a final classification at the leaf nodes.

1.The math behind Decision Tree (Entropy & Information Gain):

The algorithm selects the best feature at each split using Information Gain, which measures the reduction in entropy — a metric of impurity or uncertainty in the data — after a split, where entropy is calculated as $H = -\sum p(x) \log_2 p(x)$, and the feature producing the highest Information Gain is chosen to split the data at each node.

2. Overfitting and how it relates to Decision Trees:

A key challenge with Decision Trees is overfitting, where an unrestricted tree continues splitting until every leaf node is perfectly pure, resulting in a model that memorizes the training data rather than learning generalizable patterns, leading to high training accuracy but poor performance on unseen data.

Model 2: Random Forest

1. What is Random Forest:

Random Forest is an ensemble learning algorithm that builds multiple Decision Trees during training and combines their predictions through majority voting, addressing the overfitting problem of individual Decision Trees by introducing randomness through two key mechanisms: bootstrap sampling of the training data and random feature selection at each split.

2. Bagging explained:

The first mechanism, known as Bootstrap Aggregating or Bagging, trains each tree on a random sample of the training data drawn with replacement, meaning some websites may appear multiple times while others may not appear at all, ensuring each tree learns slightly different patterns from the data.

3. Random Feature Selection:

The second mechanism involves random feature selection, where at each split only a random subset of features is considered, calculated as the square root of the total number of features ($\sqrt{111} \approx 10$ features per split), ensuring that each tree becomes an expert in different aspects of phishing detection rather than all trees relying on the same dominant features.

4. Majority Voting:

Once all 100 trees are trained, each tree independently classifies a new website as phishing or legitimate, and the final prediction is determined by majority voting, where the class receiving the most votes across all trees is selected as the final output, making the combined prediction more robust and reliable than any single tree.

Model 3: Feedforward Neural Network

1. What is a Neural Network:

A Feedforward Neural Network is a deep learning model inspired by the structure of the human brain, consisting of interconnected layers of artificial neurons where information flows in one direction from the input layer through hidden layers to the output layer, with each neuron computing a weighted sum of its inputs and passing it through an activation function to introduce non-linearity.

2. Network Architecture:

The network architecture consists of an input layer receiving the 111 website features, followed by three hidden layers containing 128, 64, and 32 neurons respectively using ReLU activation functions to capture non-linear patterns, and an output layer with a single neuron using a Sigmoid activation function that produces a probability between 0 and 1 representing the likelihood of a website being phishing.

3. Backpropagation and Learning:

The network learns through backpropagation, where after each forward pass the prediction error is calculated using Binary Cross Entropy loss, and the gradient of the loss with respect to each of the 24,705 trainable parameters is computed and propagated backwards through the network, with the Adam optimizer adjusting all weights iteratively to minimize the prediction error.

4. Dropout and Early Stopping:

To prevent overfitting, Dropout layers with a rate of 20% were applied after the first two hidden layers, randomly deactivating neurons during training to force the network to learn robust and generalized patterns, while Early Stopping was implemented to automatically halt training when validation loss stopped improving for 5 consecutive epochs, restoring the best weights achieved during training.

Implementation

All models were implemented in Python using Google Colaboratory as the development environment, which provided free access to GPU computing resources, ensuring reproducibility and accessibility of the experimental setup.

Tools and Libraries:

1. Tools and Libraries

The implementation relied on several Python libraries: Pandas and NumPy for data loading and manipulation, Matplotlib and Seaborn for data visualization, Scikit-learn for implementing the traditional machine learning models including Decision Tree, Random Forest and TensorFlow/Keras for building and training the Feedforward Neural Network.

Experimental Setup

2. Experimental Setup

The dataset was split into 80% training (70,917 samples) and 20% testing (17,730 samples) using stratified sampling to preserve the original class distribution, and a random state of 42 was set across all models to ensure reproducibility of results, meaning any researcher running the same code will obtain identical results.

Feature Scaling:

Feature scaling was applied using StandardScaler, which transforms each feature to have a mean of zero and a standard deviation of one, and was fitted exclusively on the training data before being applied to the test data to prevent data leakage, ensuring the model had no indirect access to test set information during preprocessing.

3. Hyperparameters

Part 1: Decision Tree hyperparameters:

The Decision Tree was trained with default hyperparameters without a maximum depth restriction, allowing the tree to grow until all leaf nodes reached maximum purity, while the random state was fixed at 42 to ensure reproducibility.

Part2: Random Forest hyperparameters:

The Random Forest was configured with 100 estimators based on the principle of diminishing returns, where adding more trees beyond this point produces minimal accuracy improvement while significantly increasing computational cost, with all available CPU cores utilized in parallel through the `n_jobs=-1` parameter to reduce training time.

Part3: Neural Network hyperparameters:

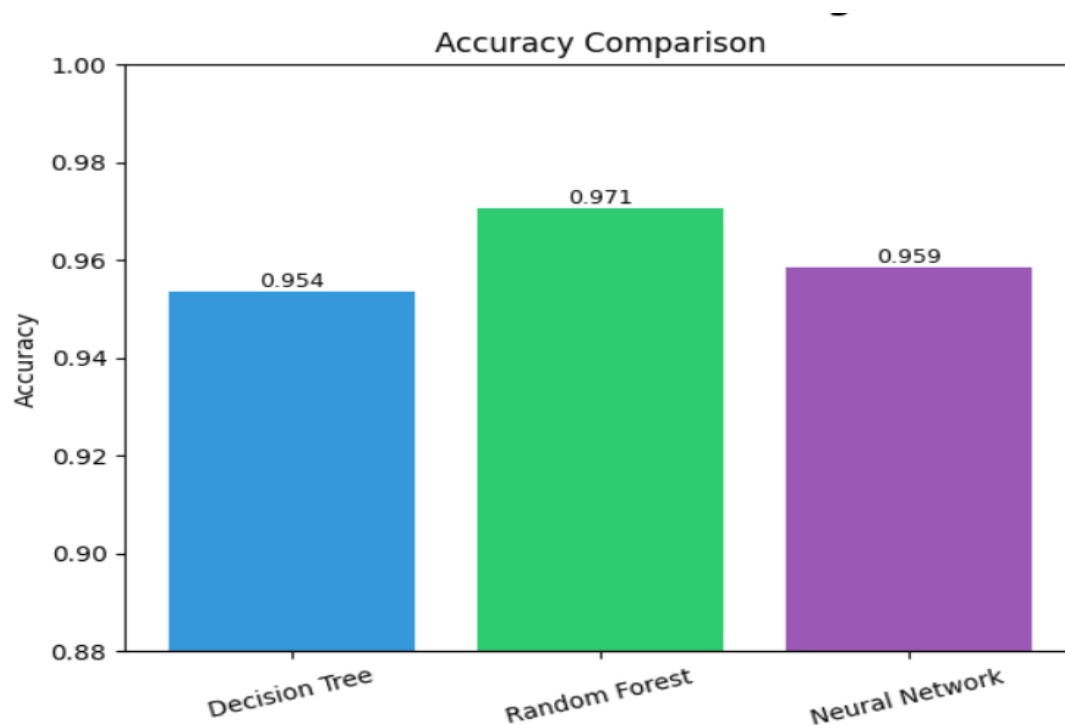
The Neural Network was trained with a batch size of 32, a maximum of 50 epochs, and a validation split of 10% carved from the training data, with the Adam optimizer selected for its ability to adaptively adjust the learning rate for each parameter individually, and Early Stopping with a patience of 5 epochs implemented to automatically halt training when validation loss stopped improving, restoring the best weights achieved during training.

Part4 : Results & Analysis

All three models were evaluated on the same held-out test set of 17,730 websites using four key metrics: accuracy, precision, recall, and F1-score, with particular emphasis on phishing recall as the most critical metric in a security context where missing a phishing website poses a direct risk to users.

Experience 1

The plot summarizes the overall accuracy achieved by each model on the test set, with Random Forest achieving the highest accuracy of 97.06%, followed by Neural Network at 96.20%, Decision Tree at 95.36%, demonstrating that ensemble methods outperform both linear and deep learning approaches on this structured tabular dataset.



Precision, Recall, F1 Comparison

Precision, Recall and F1-Score Comparison

experience 2:

The table presents the precision, recall and F1-score for each model specifically for the phishing class, with Random Forest achieving the highest recall of 96%, meaning it correctly identified 96% of all actual phishing websites in the test set, indicating it missed the highest proportion of real phishing websites.

Model	Precision	Recall	F1-Score
Decision Tree	93%	93%	93%
Random Forest	96%	96%	96%
Neural Network	95%	94%	95%

Confusion Matrices Analysis

experience 3:

The confusion matrices revealed that Random Forest achieved the lowest number of False Negatives with only 249 missed phishing websites, compared to 355 for Neural Network, 413 for Decision Tree, confirming that Random Forest is the most reliable model for real-world phishing detection where missing a phishing website directly exposes users to credential theft.

Model	TN	FP	FN	TP
Decision Tree	11,191	409	413	5,717
Random Forest	11,328	272	249	5,881
Neural Network	11,283	317	355	5,775

Training Time Comparison

experience 4:

The plot compares the training time of all models, revealing that the Neural Network was by far the slowest model at 188 seconds, approximately 13 times slower than Random Forest at 13.9 seconds, while Decision Tree was the fastest at approximately 1 second, demonstrating that deep learning carries a significant computational cost that is not justified by its performance on this structured tabular dataset.



Feature Importance

Part 1:

The Random Forest model provided insight into the most influential features for phishing detection, identifying directory length, time of domain activation, and the quantity of dollar signs in the directory as the three most important features, suggesting that phishing websites are most distinguishable from legitimate ones through the length and complexity of their URL directory paths and the recency of their domain registration.

Part 2:

These findings are consistent with known phishing behavior, as attackers typically register domains shortly before launching attacks and use long, complex URL paths to disguise the true destination of their malicious websites, validating the feature engineering approach used in the dataset.

Discussion

The results of this study provide several important insights into the effectiveness of different machine learning approaches for phishing website detection, highlighting the relationship between model complexity, dataset characteristics, and predictive performance.

Question: Why Random Forest Won ?

Random Forest as the Best Performing Model

Part 1:

Random Forest outperformed all other models across every evaluation metric, achieving the highest accuracy of 97.06%, the highest phishing recall of 96%, and the lowest number of missed phishing websites at 249, which can be attributed to its ensemble nature where 100 decision trees collectively correct each other's individual errors through majority voting, producing a more robust and generalized model than any single algorithm.

Part 2:

Furthermore, Random Forest's ability to naturally handle non-linear relationships between features through its tree-based splitting mechanism proved particularly well suited to the complex patterns present in phishing website data, where the relationship between features such as directory length, domain activation time, and phishing behavior cannot be captured by a simple linear boundary.

Question: Why Neural Network Didn't Beat Traditional ML ?

Deep Learning Performance on Structured Tabular Data

Part 1:

Despite the widespread success of deep learning in many domains, the Feedforward Neural Network failed to outperform Random Forest on this dataset, achieving a lower accuracy of 96.20% while requiring 188 seconds of training time compared to Random Forest's 13.9 seconds, suggesting that deep learning does not always outperform traditional machine learning approaches, particularly on structured tabular data with engineered features.

Part 2:

Neural Networks excel at automatically discovering features from raw unstructured data such as images, text, and audio, however since the features in this dataset were already carefully engineered by domain experts to capture the most relevant phishing indicators, the Neural Network had no advantage in feature discovery, allowing Random Forest to leverage these well-crafted features more efficiently through its tree-based splitting mechanism.

Limitations of the Project

Limitations

Part 1:

Despite the strong results achieved, this project has several limitations that should be acknowledged, including the use of a static dataset that may not reflect the constantly evolving nature of phishing attacks, as attackers continuously develop new techniques that may not be captured by the features present in the dataset.

Part2 :

Finally, while the dataset contains 111 carefully engineered features, real-world phishing detection systems may require additional dynamic features such as website content analysis, JavaScript behavior, and real-time domain reputation checks to achieve even higher detection rates and adapt to emerging phishing techniques.

Conclusion

1. Summary of Findings

This project successfully developed and compared three machine learning models for phishing website detection, demonstrating that Random Forest achieved the best overall performance with 97.06% accuracy and 96% phishing recall, outperforming Decision Tree at 95.36%, Neural Network at 96.20%, establishing that ensemble tree-based methods are the most effective approach for structured phishing detection datasets.

2. Deep Learning conclusion:

The comparison between traditional machine learning and deep learning revealed that a Feedforward Neural Network, despite its greater complexity and significantly higher computational cost of 188 seconds, failed to surpass the Random Forest model, confirming that deep learning does not universally outperform traditional machine learning and that the nature of the data plays a critical role in determining the most effective approach.

Real World Implications

The results of this study have direct real-world implications for cybersecurity systems, as a Random Forest model achieving 97.06% accuracy and missing only 249 phishing websites out of 6,130 could be deployed as an automated browser extension or web proxy to protect users from phishing attacks in real time, significantly reducing the risk of credential theft and financial fraud.

Future Work

1. Thought 1

Future work could explore several directions to improve upon the results of this study, including experimenting with more advanced deep learning architectures such as Long Short-Term Memory networks or Transformer models that may better capture sequential patterns in URL structures, as well as incorporating dynamic features such as real-time domain reputation, website content analysis, and JavaScript behavior analysis to improve detection of emerging phishing techniques.

2. Thoughts 2

Additionally, future research could address the class imbalance present in the dataset using techniques such as SMOTE (Synthetic Minority Oversampling Technique) to generate synthetic phishing samples, potentially improving the recall of all models, as well as exploring real-time retraining mechanisms that continuously update the model with newly discovered phishing websites to keep up with the constantly evolving nature of phishing attacks.

References

Colab link : [phishing_detection_update.ipynb - Colab](#)

APWG. (2024). Phishing Activity Trends Report, 4th Quarter 2024. Anti-Phishing Working Group.
<https://apwg.org/trendsreports/>

Aleroud, A., & Zhou, L. (2017). Phishing environments, techniques, and countermeasures: A survey. *Computers & Security*, 68, 160-196.
<https://www.sciencedirect.com/science/article/pii/S1084804517300590>

Mohammad, R., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*.

[https://archive.ics.uci.edu/dataset/327/
phishing+websites](https://archive.ics.uci.edu/dataset/327/phishing+websites)

Alhogail, A. (2018). Applying machine learning to phishing email detection. International Journal of Computer Applications.

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
<https://www.deeplearningbook.org/>

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), 1929-1958.

Vrbancic, G. (2020). Phishing Dataset. GitHub Repository.
<https://github.com/GregaVrbancic/Phishing-Dataset>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

<https://scikit-learn.org>

Abadi, M., et al. (2016). TensorFlow:

A system for large-scale machine learning.

USENIX Symposium on Operating Systems

Design and Implementation.

<https://www.tensorflow.org>